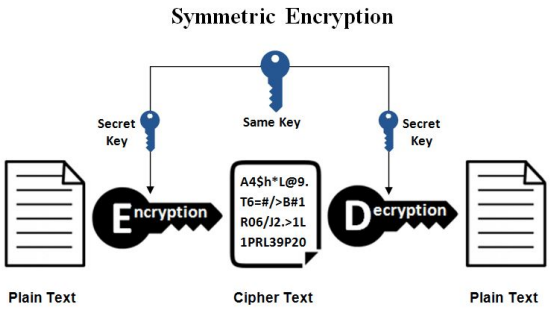
# Cryptographie symétrique

Projet S3 - 2020 - 2021




## Table des matières

**1 Introduction** **2**

**2 Partie 1 : Chiffrer/déchiffrer un message avec une clef symétrique** **2**
2.1 Méthode 1 : Chiffrage caractère par caractère avec l'opérateur *xor* . . . . . 2
2.2 Travail à réaliser . . . . . 3
2.3 Méthode 2 : Chiffrage par blocs : Cypher Block Chaining (CBC) . . . . . 3
2.4 Travail à réaliser . . . . . 5

**3 Partie 2 : Le problème de l'échange de la clef secrète : échange de Diffie-Hellman** **5**
3.1 Principe d'un échange de clés Diffie-Hellman dans $Z/pZ$ avec $p$ premier . . . . . 6
3.2 Travail à réaliser . . . . . 6
3.3 Séquence culturelle (ou apocalyptique suivant les avis) : Fondement mathématique 7

**4 Partie 3 : Cracker le code** **8**
4.1 Attaque sur le message chiffré . . . . . 8
4.1.1 Crack C1 : émondage des caractères possible pour la clef . . . . . 8
4.1.2 Crack C2 : statistiques des lettres dans la langue cible . . . . . 9
4.1.3 Crack C3 : utilisation d'un dictionnaire de la langue cible . . . . . 10
4.1.4 Travail à faire . . . . . 10
4.2 Attaque sur la clef : Pohlig-Hellman . . . . . 11

**5 Evaluation** **11**
5.1 Organisation globale du code et points associés (12 points) . . . . . 11
5.2 Livrables de la partie Gestion de projet (8 pts) . . . . . 11

**6 Tests, commentaires, livrables** **12**
6.1 Prévisionnel . . . . . 12

# 1 Introduction

La cryptographie consiste à remplacer un message lisible par tout le monde, le message en clair, par un message inintelligible (le chiffré) sauf pour ceux qui connaissent le "truc" qui permet de retrouver le message en clair.

Le "truc" consiste à utiliser une clef qui permet de transformer le message en clair en message codé (opération de chiffrage) et vice-verça (opération de déchiffrage). Si la clef de chiffrage et de déchiffrage est la même on parle de chiffrage symétrique [1]. La clef doit rester secrète et partagées seulement entre les personnes voulues.

Chercher à déchiffrer un message codé, donc chiffré, sans en avoir la clef est le décryptage. On parle de "casser le code", c'est à dire trouver la clef. On emploie aussi le terme anglais crack (to crack the code, cracker le code).

Le projet traite de cryptographie symétrique et est divisé en trois parties indépendantes mais complémentaires.

— Indépendantes veut dire qu'on peut programmer chaque partie sans avoir rien fait concernant les autres.
— Complémentaires veut dire que les trois parties participent à la construction d'un système complet de génération de clef, chiffrage/déchiffrage et de décryptage (restreint).

Dans les différentes parties, le travail à réaliser entre dans une ou plusieurs de ces catégories :

— intégrer un code fourni dans une nouvelle fonctionnalité
— développer un code à partir de prototypes et de spécifications
— implémenter un algorithme ou une méthode
— comprendre un problème, imaginer une solution et la programmer

**Dans la suite on présente le problème et le travail à réaliser pour chacune des parties. Tout n'est pas à faire, il vous faudra choisir ce que vous aller programmer dans votre prévisionnel (voir section 6)**.

# 2 Partie 1 : Chiffrer/déchiffrer un message avec une clef symétrique

Cette partie s'occupe de chiffrer et déchiffrer des messages. On va utiliser deux méthodes différentes. Le message en clair (à chiffrer) sera lu dans un fichier texte, et le message chiffré (le chiffré) sera écrit dans un autre fichier texte.

## 2.1 Méthode 1 : Chiffrage caractère par caractère avec l'opérateur *xor*

La façon la plus simple (et la moins robuste) pour chiffrer un message est d'utiliser l'opérateur *xor* de ou exclusif(^en C). L'opérateur va faire le xor bit à bit de ses deux opérandes. Les opérandes

1. Il existe des chiffrages asymétriques.

ici sont des *char* donc des octets.

**Remarque :** il est préférable de raisonner en terme d'octets plutôt que de caractères. En effet ce procédé de chiffrage peut s'appliquer sur n'importe quel fichier de données (image, son, exécutable, etc) en traitant le fichier octet par octet.

On peut définir un type byte : typedef unsigned char byte ;

**Principe de la méthode :** Soit *msg* un message (char * ou byte *) [2] et *key* une chaîne de caractères (char * ou byte *). Le principe est d'écrire la clef sous le message et de faire le *xor* des octets en colonnes. Si la clef est plus courte que le message on répète la clef autant de fois que nécessaire. Si la clé est plus longue que le message, on la tronque à la longueur du message. Ensuite on fait le xor sur les codes UTF8 des caractères (en C on peut faire directement $c1\hat{}c2$ avec $c1$ et $c2$ de type *char*).

**Exemple :** msg = "Les carottes sont cuites", key = "rutabaga"

Chiffrage :

| L | e | s | | c | a | r | o | t | t | e | s | | s | o | n | t | | c | u | i | t | e | s |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| r | u | t | a | b | a | g | a | r | u | t | a | b | a | g | a | r | u | t | a | b | a | g | a |
| 3e | 10 | 7 | 41 | 1 | 0 | 15 | e | 6 | 1 | 11 | 12 | 42 | 12 | 8 | f | 6 | 55 | 17 | 14 | b | 15 | 2 | 12 |

Le résultat du chiffrage n'étant pas obligatoirement formé de caractères affichables, il est ici écrit en hexadécimal.

Le déchiffrage consiste simplement à faire la même opération en partant du message chiffré.

## 2.2 Travail à réaliser

1. Ecrire une fonction implémentant le chiffrage XOR sur les messages et des clefs de type *char ** ou *byte **. Le message en clair sera lu dans un fichier texte et le chiffré sera écrit dans un autre fichier. Les noms des fichiers et la clef seront passés en paramètre de la fonction.
2. Ecrire un programme principal de test et vérifier l'opération de chiffrement, puis de déchiffrement en fournissant comme fichier d'entrée le message chiffré. Bien sûr on doit retrouver le message d'origine.

## 2.3 Méthode 2 : Chiffrage par blocs : Cypher Block Chaining (CBC)

La méthode précédente est trop facile à cracker car elle induit trop de régularités dans le chiffré. Nous allons utiliser une autre méthode basée elle aussi sur l'opérateur xor.

A lieu de chiffrer les caractères (ou octets) du message un par un, on va traiter plusieurs octets à la fois. Le groupe d'octets traité à chaque étape s'appelle un bloc. Le nombre d'octets constituant un bloc est fixe et choisi au départ. Chaque bloc donnera un bloc chiffré de même taille.

Le message est donc divisé en une suite $B_1, B_2, \ldots, B_n$ de blocs de même taille. Le dernier bloc $B_n$ peut être éventuellement complété avec des caractères espace si besoin. On va donc chiffrer les blocs dans l'ordre $B_1$, jusqu'à $B_n$ pour produire une suite de blocs chiffrés $C_1, ..., C_n$.

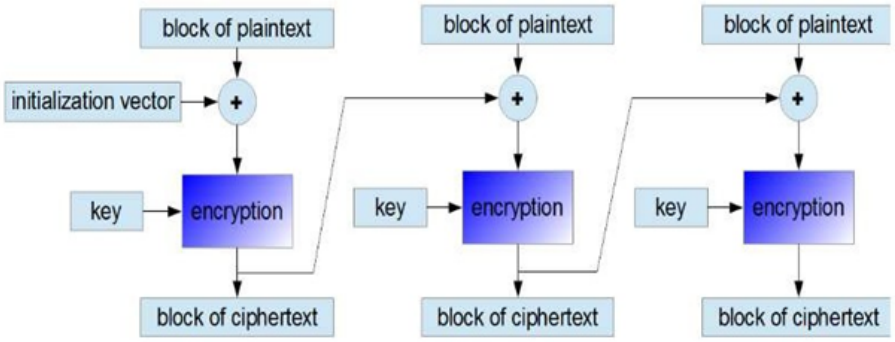
La méthode CBC consiste, pour chiffrer le bloc $B_i$, à :

2. En C les chaînes de caractères sont des tableaux dynamiques.

1. Faire un *xor* entre le bloc chiffré $C_{i-1}$ et le bloc clair $B_i$ pour produire le bloc intermédiaire $CC_i$,
2. Chiffrer $CC_i$ avec la méthode de chiffrement choisie, $\mathcal{E}_K$ où $K$ est la clef, pour obtenir $C_i$.

Pour commencer le processus, c'est à dire chiffrer le premier bloc pour lequel il n'y a pas de chiffré d'ordre précédent, on utilise un *vecteur d'initialisation* (VI) qui est un bloc arbitraire choisi au départ.

$C_1 = \mathcal{E}_K(B_1 \oplus VI)$
$C_i = \mathcal{E}_K(B_i \oplus C_{i-1})$
où $\oplus$ symbolise l'opérateur *xor*.

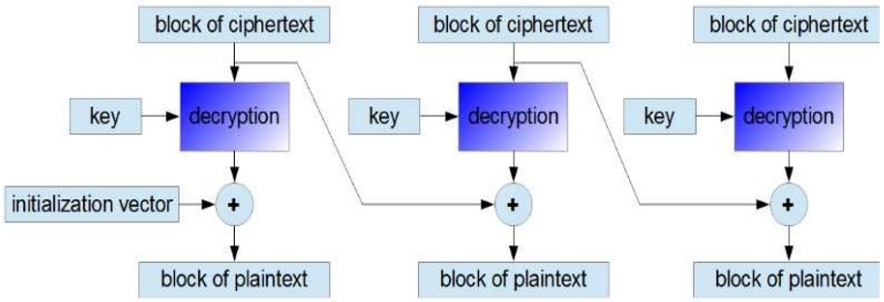



*Cryptage en mode CBC*

Le déchiffrement se fait en renversant le processus :

$B_1 = VI \oplus \mathcal{D}_K(C_1)$
$B_i = C_{i-1} \oplus \mathcal{D}_K(C_i)$

où $\mathcal{D}_K$ est la fonction de déchiffrement indicée par $K$, la clef.




*Décryptage en mode CBC*

### 2.4 Travail à réaliser

1. Implémenter la méthode CBC avec des blocs de 16 octets, un vecteur d'initialisation passé en paramètre qui devra faire lui aussi 16 octets (caractères [3]), et comme méthode de chiffrement/déchiffrement un simple *xor*, c'est à dire que $\mathcal{E}_K = \mathcal{D}_K = xor$ [4] avec une clé char * (ou byte *), c'est à dire un type chaîne de caractères en C [5]. Les méthodes de chiffrement et de déchiffrement étant différentes dans leurs protocoles, l'implémentation consistera en deux fonctions : une pour chiffrer et une pour déchiffrer.
2. Créer un programme principal nommé *dh_crypt*, appelant la fonction implémentant la méthode 1 (avec xor) et la méthode 2 (les fonctions CBC), et admettant comme paramètres de la ligne de commande les options :
   — -i suivie du nom du fichier contenant le message à chiffrer
   — -o suivie du nom du fichier où l'on va écrire le chiffré
   — -k suivie de la clef
   — -m suivie de la méthode de chiffrement : xor ou cbc-crypt suivie du vecteur d'initialisation, ou cbc-uncrypt suivie également du vecteur d'initialisation
   — -h affichant l'aide des commandes. Cette option, si elle est présente, annule toutes les autres.

   Ces options (ou balises) pourront être écrire dans un ordre quelconque.

   **Exemple :**

```
dh_crypt -i clair.txt -o crypt.txt -k rutabaga -m xor
dh_crypt -o crypt.txt  -i clair.txt -m xor  -k rutabaga
dh_crypt -i crypt.txt -o uncrypt.txt -k rutabaga -m xor
dh_crypt -i clair.txt -o crypt.txt -k rutabaga -m cbc-crypt azertyuiopqsdfgh
dh_crypt -m cbc-uncrypt azertyuiopqsdfgh -i crypt.txt -o uncrypt.txt -k rutabaga
dh_crypt -h
```

# 3 Partie 2 : Le problème de l'échange de la clef secrète : échange de Diffie-Hellman

La cryptographie par clef symétrique est simple mais demande que les agents communiquant utilisent la même clef. Ils ont pu se mettre d'accord au préalable, en se rencontrant par exemple. Mais s'ils sont à distance et ne peuvent se mettre d'accord sur la clef sans être espionnés, comment faire ?

Whitfield Diffie et Martin Hellman, ont imaginé une méthode, publiée en 1976, par laquelle deux agents, nommés par convention Alice et Bob, peuvent se mettre d'accord sur un nombre (qu'ils peuvent utiliser comme clé pour chiffrer une conversation ou un message) sans qu'un troisième agent, appelé Ève puisse découvrir ce nombre, même en ayant écouté tous leurs échanges. Cette idée valut en 2015 aux deux auteurs le prix Turing (dixit Wikipédia).

Pour implémenter cette méthode, il nous faut un corps fini et un nombre premier $p$. Par exemple $Z/pZ$, l'ensemble des nombres relatifs $\{-(p-1), \ldots, -1, 0, 1, \ldots, p-1\}$

3. Il vaut mieux ici prendre des caractères non accentués pour avoir un octet par caractère en codage UTF8.
4. C'est à dire $\mathcal{E}_K(m) = m$ xor $K$, et même chose pour $\mathcal{D}_K$.
5. Les chaînes de caractères en C sont des tableaux dynamiques de caractères.

## 3.1 Principe d'un échange de clés Diffie-Hellman dans $Z/pZ$ avec $p$ premier

**Remarque :** La notation $[p]$ signifie *modulo* $p$

1. Alice et Bob ont choisi un groupe fini $(Z/pZ)$ et un générateur $g$ de ce groupe. **En clair ils choisissent $p$ premier, calculent $g$ et utilisent les nombres de 0 à $p-1$. Tous tous les calculs modulo $p$.**
2. Alice choisit un nombre $a$ au hasard, $(1 \leq a \leq p-1)$ élève $g$ à la puissance $a$, et communique à Bob ce nombre $A = g^a$ (calculé modulo $p$), comme montré dans l'exemple ci-dessous), c'est-à-dire le nombre $A$.
3. Bob fait de même avec le nombre $b$ choisi au hasard $(1 \leq b \leq p-1)$. Il transmet à Alice le nombre $B = g^b[p]$ ;
4. Alice, en élevant le nombre B reçu de Bob à la puissance a, obtient $(g^b)^a[p]$.
5. Bob fait le calcul analogue avec le $A$ reçu d'Alice et obtient $(g^a)^b$, qui est le même résultat que celui obtenu par Alice en 4., soit la clé commune aux deux.
6. L'astuce est qu'il est difficile d'inverser l'exponentiation dans un corps fini, c'est-à-dire de calculer le logarithme discret (trouver $a$ tel que $h = g^a$ connaissant $h$ et $g$), Ève ne peut découvrir ni $a$ ni $b$, donc ne peut pas calculer $(g^a)^b[p]$ (à condition que le nombre $p$ soit choisi assez grand). Finalement, Alice et Bob connaissent donc tous les deux le nombre $(g^a)^b[p]$ dont Ève n'a pas connaissance.

**Exemple :**

1. Alice et Bob ont choisi un nombre premier $p$ et une base $g$. Par exemple, $p = 23$ et $g = 5$
2. Alice choisit un nombre secret $a = 6$
3. Elle envoie à Bob la valeur $A = g^a[p] = 5^6[23] = 15625[23] = 8$
4. Bob choisit à son tour un nombre secret $b = 15$
5. Bob envoie à Alice la valeur $B = g^b[p] = 5^{15}[23] = 19$
6. Alice peut maintenant calculer la clé secrète : $B^a[p] = 19^6[23] = 2$
7. Bob fait de même et obtient la même clé qu'Alice : $A^b[p] = 8^{15}[23] = 2$

**Remarque importante :** Dans la pratique, on prend un nombre premier $p$ dit de Sophie Germain (tel que $q = 2p + 1$ premier lui aussi) de grande taille et un générateur $g$ dans $Z/pZ$ ($g$ est donc premier avec $p-1$). Cela évite de choisir un nombre $p = 2q + 1$ avec $q$ non premier et décomposable en une suite de facteurs. On a en effet des méthodes pour calculer $p$ à partir des facteurs et de réduire ainsi la puissance de calcul nécessaire.

## 3.2 Travail à réaliser

1. Ecrire une fonction de simulation en C du protocole d'échange de clefs de Diffie-Hellman, c'est à dire de la construction de la clef commune par Alice et Bob par échange d'information sur un réseau. Les échanges sur le réseau seront simulés par des affichages (printf) des envois entre les deux agents. Les paramètres $p$, le nombre premier et $g$ le générateur seront des paramètres de la fonction.
   On utilisera la fonction :

   ```
   long puissance_mod_n (long a, long e, long n)
   ```

calculant la puissance modulaire, $a^e[n]$

La simulation devra préciser ce qui est connu d'Alice seulement, de Bob seulement, des deux et de Eve (l'espion). On affichera explicitement ce qui est transmis sur le réseau et ce qui est calculé en privé.

**Remarque :** Normalement nous devrions faire les calculs en utilisant une bibliothèque permettant de faire des calculs en précision illimitée comme *GMP*. Pour simplifier ce projet nous nous limiterons à les *long*. Donc attention aux dépassements de capacité.

2. Ecrire un programme principal *dh_genkey* permettant de générer un nombre premier de Sophie Germain, un générateur, et de lancer la simulation d'échange de la clef commune et admettant les options :
   — -o suivie d'un nom de fichier où les résultats seront écrits : $p$ le nombre premier généré, $g$ le générateur, le reste de la simulation ($a$, $b$ et les échanges) étant écrits à l'écran. Si cette option n'est pas présente, les résultats sont tous écrits à l'écran.

   — -h donnant la syntaxe d'exécution sans autre action.

   Il s'agit d'automatiser les opérations de création de $p$ et de $g$ en appelant les fonctions suivantes :
   — Calcul du nombre premier $p$ (de Sophie Germain) avec la fonction :
   *long genPrimeSophieGermain(long min,long max,int *cpt)*
   — Calcul du générateur : *long seek_generator(long start,long p)*
   — Choix de $a$ et $b$ avec *long random_long(long min,long max)*

   **Remarque :** Les fonctions cités sont données dans le fichier *dh–prime.c* sur Moodle. Des commentaires Doxygen sont suffisants pour en comprendre l'usage et l'action.

**Exemple :**

```
dh_genkey
dh_genkey -o mykey.txt
dh_genhey -h
```

## 3.3 Séquence culturelle (ou apocalyptique suivant les avis) : Fondement mathématique

La méthode utilise la notion de groupe (multiplicatif), par exemple celui des entiers modulo $p$, où $p$ est un nombre premier : dans ce cas, les opérations mathématiques (multiplication, puissance, division) sont utilisées telles quelles, mais le résultat doit être divisé par $p$ pour ne garder que le reste, appelé modulo. Les groupes ayant la propriété de l'associativité, l'égalité $(g^b)^a = (g^a)^b$ est valide et les deux parties obtiennent bel et bien la même clé secrète.

La sécurité de ce protocole réside dans la difficulté du problème du logarithme discret : pour que Ève retrouve $(g^a)^b$ à partir de $g^a$ et $g^b$, elle doit élever l'un ou l'autre à la puissance $b$ ou à la puissance $a$ respectivement. Mais déduire $a$ (resp. $b$) de $g^a$ (resp. $g^b$) est un problème que l'on ne sait pas résoudre efficacement. Ève est donc dans l'impossibilité (calculatoire) de déduire des seules informations $g^a$, $g^b$, $g$ et $p$, la valeur de $(g^a)^b$ .

Il faut toutefois que le groupe de départ soit bien choisi et que les nombres utilisés soient suffisamment grands pour éviter une attaque par recherche exhaustive. À l'heure actuelle, un

nombre premier $p$ de l'ordre de 300 chiffres ainsi que $a$ et $b$ de l'ordre de 100 chiffres sont tout simplement impossibles à casser [6] même avec les meilleurs algorithmes de résolution du logarithme discret.

# 4 Partie 3 : Cracker le code

Bon, tout ça est très rigolo mais ce que tout le monde attend, c'est de pouvoir cracker un code. Nous y sommes. Vu que cela n'est pas si simple nous allons nous limiter au cas où le message a été chiffré à l'aide d'une méthode simple, c'est à dire le *xor* sur chaque caractère (Partie 1, méthode 1).

## 4.1 Attaque sur le message chiffré

Le processus par lequel on tente de décoder un message est appelé une attaque. Le chiffrage et le déchiffrage d'un fichier peut se faire sans se préoccuper de la nature des octets. Par contre, pour casser un chiffre nous devons connaître la langue dans laquelle le message a été écrit, ainsi que le codage informatique de ses caractères.

Le codage informatique utilisé sur la plupart des ordinateurs récents est le UTF-8. C'est un codage qui utilise entre un et quatre octets pour coder un caractère et qui est prévu pour représenter tous les caractères des langues de la planète. Il reprend le codage ASCII sur sept bits mais pour les caractères européens (et donc les caractères accentués du français), il utilise deux octets dont le premier (C3 en hexadécimal) sert de balise.

Du coup il faut faire un choix entre les deux options suivantes :

— Soit nous considérons que le texte en français est codé selon la norme Latin-1 sur un octet et non en UTF-8. Il faudra donc convertir ou enregistrer les textes à cette norme, ce qui est prévu dans la plupart des éditeurs de texte.

— Soit nous considérons que le texte est écrit en anglais en UTF8. Pas d'accent, pas de problème.

Dans la suite nous allons considérer que nous décryptons un texte écrit en anglais en UTF8 [7] car cela simplifie énormément les choses. Si vous voulez traiter le cas de texte écrits en français et codés selon la norme Latin-1, contactez votre tuteur.

Nous présentons dans cette sections trois types d'attaque : C1, C2, C3. Nous allons les utiliser de manière hiérarchique C1, puis C2, puis C3. Ces attaques partent du principe que le message chiffré est un texte écrit en anglais. Pour toutes ces attaques, on va faire une liste des clefs possibles, dites clefs candidates, et chercher par différents moyens à éliminer les clefs impossibles. En dernier ressort, il faudra essayer chacune des clefs restantes pour décrypter le chiffré.

**Remarque importante :** Toujours dans un souci de simplification, nous allons considérer que la clef est une chaîne de caractères composée uniquement de caractères numériques ('0' à '9').

### 4.1.1 Crack C1 : émondage des caractères possible pour la clef

L'ensemble des lettres de l'anglais est représenté par un sous ensemble des octets codés sur 8 bits. Une première attaque qui permet de trouver un ensemble de clés valides, consiste à vérifier que les caractères décodés par une clé candidate sont valides dans la langue et le codage choisis. En clair, si $x$ est un caractère du chiffré, et $y$ un caractère de la clef (inconnue) qui a peut-être

6. Pour l'instant en 2020.

7. Le codage des caractères de la norme ASCII est le même dans la norme UTF8. On parlera d'ASCII dans la suite.

été utilisé pour chiffrer le texte alors $z = x$ xor $y$ doit redonner le caractère d'origine, c'est à dire que le code de $z$ doit correspondre à un caractère ASCII admissible, c'est à dire, un caractère de la langue anglaise, un chiffre ou un caractère de ponctuation.

On ne validera pas d'une manière globale chaque clef candidate, ce serait trop long. On validera l'un après l'autre chaque caractère valide des clefs. Précisément si la clef a pour longueur 3, le caractère clef[0] sert à coder les caractères $1, 4, ..., 1 + 3k, ...$ du message d'origine. On détermine pour commencer les valeurs possibles de clef[0] puis, de manière tout à fait indépendante, on déterminera les valeurs possibles de clef[1] et de clef[2].

On va considérer tous les caractères possibles pour chaque clef[i], $0 \leq i <$ longueur de la clé. Par exemple, si le chiffré est msg = "s(/1&!" et que la clef a trois caractères, alors clef[0] a servi à chiffrer msg[0] et msg[3]. Pour chaque valeur $c$ candidate pour clef[0] on fait : msg[0] xor c = 's' xor c qui doit donner une valeur admissible de la table ASCII. De même, msg[3] xor c = '1' xor c qui doit aussi donner une valeur admissible de la table ASCII.

On va mémoriser l'ensemble des caractères admissibles pour chaque caractère de la clef. De là on peut établir la liste des clefs candidates en faisant une sorte de produit cartésien des listes de caractères.

Par exemple, si on obtient :
- $clef[0] \in ['5', '6', '8']$
- $clef[1] \in ['1']$
- $clef[2] \in ['0', '2']$
- alors on aura aisément la liste des $3 * 1 * 2 = 6$ clefs selon C1 :

$$clef \in ["510", "512", "610", "610", "810", "812"]$$

.

Les caractères valides pour la clef peuvent être stockés dans un tableau statique ou dynamique à deux dimensions *clef[tailleClef][maxCaracteres]* ou *tailleClef* est la taille de la clef recherchée, et *maxCaracteres* le nombre de caractères possible pour un caractères de la clef. Comme, dans cette partie, nous nous limitons à des clefs formées de caractères numériques, *maxCaractères* est égal à 10 (caractères de 0 à 9).

Au départ, chaque ligne du tableau (chaque caractère de la clef) contient tous les caractères possibles, c'est à dire les caractères numériques de '0' à '9', sauf pour le premier caractère ou '0' est exclu. Au fur et à mesure des tests, les caractères impossibles sont supprimés.

### 4.1.2 Crack C2 : statistiques des lettres dans la langue cible

Dans notre chiffrement caractère par caractère, une même lettre risque d'être chiffrée par le même nombre. L'analyse fréquentielle examine les répétitions des codes du message chiffré afin de trouver la clef. Elle est principalement utilisée contre les chiffrements mono-alphabétiques (substitution d'une lettre par une autre) qui présentent un biais statistique. Elle est moins pertinente pour un chiffrage par clef mais reste utile.

L'analyse fréquentielle est basée sur le fait que, dans chaque langue, certaines lettres ou combinaisons de lettres, apparaissent avec une certaine fréquence. Par exemple, en français, le e est la lettre la plus utilisée, suivie du a et du s. Inversement, le w est peu usité.

Le lien ci-dessous donne les fréquences pour diverses langues. On utilisera la colonne correspondant à l'anglais.

`https://en.wikipedia.org/wiki/Letter_frequency`

Le tableau peut être téléchargé sur Moodle.

Dans ce tableau, les diverses formes des lettres sont prises en compte : par exemple pour le français et pour la lettre 'E' les formes 'e','E','é','è' et 'ê' sont prises en compte. Cet aspect est sans objet pour les textes en langue anglaise.

Avec chaque clef candidate obtenue par la méthode C1, on décode le cryptogramme et on calcule par comptage les diverses fréquences d'apparitions des lettres du texte décodé (mais pas forcément déchiffré) avec la clef. On obtient alors un tableau de fréquences noté $Freq$. La meilleure clef est celle pour laquelle les fréquences mesurées sont les plus proches des fréquences théoriques dans la langue cible : $Freq_th$. Cette notion de proximité et donc de distance entre deux tableaux sera mesurée par la fonction $d$ définie par :

$$d(Freq_th, Freq) = \sum_{i=1}^{26} (Freq_th[i] - Freq[i])^2$$

Cette répartition des fréquences des lettres n'est qu'approximative, cela dépend de nombreux paramètres tels que le niveau de langue du texte, ainsi que du style d'écriture. Une deuxième condition nécessaire pour appliquer cette technique est la longueur du cryptogramme. En effet, un texte trop court ne reflète pas obligatoirement la répartition générale des fréquences des lettres.

Quelquefois, on peut constater que la clef sélectionnée par cette méthode n'est pas la bonne. Cela provient du manque de représentativité du texte : trop petite taille, style trop particulier. On s'oriente alors vers une méthode utilisant un dictionnaire.

On cherche la clef avec le meilleur score. Pour cela, à partir des clefs valides pour C1, on "décode" les textes et on calcule les fréquences. On retient la clef avec les caractères offrant le meilleur score en suivant l'analyse fréquentielle. L'ensemble des clefs sera mémorisé dans un tableau et classé par ordre de score décroissant. Si plusieurs clés obtiennent le même score, on les classe entre elles avec un critère arbitraire (ordre lexicographique par exemple).

### 4.1.3 Crack C3 : utilisation d'un dictionnaire de la langue cible

Cette méthode consiste à chercher la clef pour laquelle le texte décodé contient exclusivement des mots présents dans un dictionnaire. Cela suppose qu'on dispose d'un dictionnaire contenant toutes les formes nominales et verbales ; c'est le cas du dictionnaire du scrabble qui vous est fourni. Si on suppose le dictionnaire complet et le message écrit sans faute, la bonne clef est celle pour laquelle tous les mots du message décodé sont dans le dictionnaire.

A partir de la liste des clefs valides calculée avec C1 et C2, on décode les textes et on cherche chaque mot du texte "décodé" dans le dictionnaire fourni [8]. On compte le nombre de mots valides suivant le dictionnaire et on classe les clefs par ordre décroissant de ce nouveau score.

**Nota bene :** On suppose que seule la première lettre d'un mot du message peut éventuellement être une majuscule. Pour chercher ce mot dans le dictionnaire il faudra remplacer cette majuscule par une minuscule.

### 4.1.4 Travail à faire

1. Créez les fonctions *dh_crack_c1.c*, *dh_crack_msg_c2.c*, e *dh_crack_msg_c3.c* implémentant chacune des recherches précédentes.
2. Créez un programme *dh_crack* permettant de lancer les fonctions ci-dessus, dans l'ordre, avec les options :

8. Le dictionnaire fourni est sans aucune majuscule.

-i fichier-à-décrypter
-m c1 ou all (c1 lance la méthode c1 seulement)
-k longueur-de-la-clef
-d dictionnaire (cette option n'est utilisée que pour -m all qui lance les trois méthodes c1, c2, c3 dans cet ordre).
-h donnant l'aide sans autre action
*dh_crack* affichera pour chaque clef candidate, son score en nombre de mots présents dans le dictionnaire.

**Exemple :**

```
./dh_crack -i Datas/Crypted/ringCxor.txt -m c1 -k 4
./dh_crack -i Datas/Crypted/ringCxor.txt -m all -k 4 -d Dicos/english.txt
```

**Remarque :** Le fichier *ctype.h* propose des fonctions permettant de tester si un caractère est alphanumérique, une ponctuation ou autre. Utilisez ces fonctions plutôt que de les refaire.

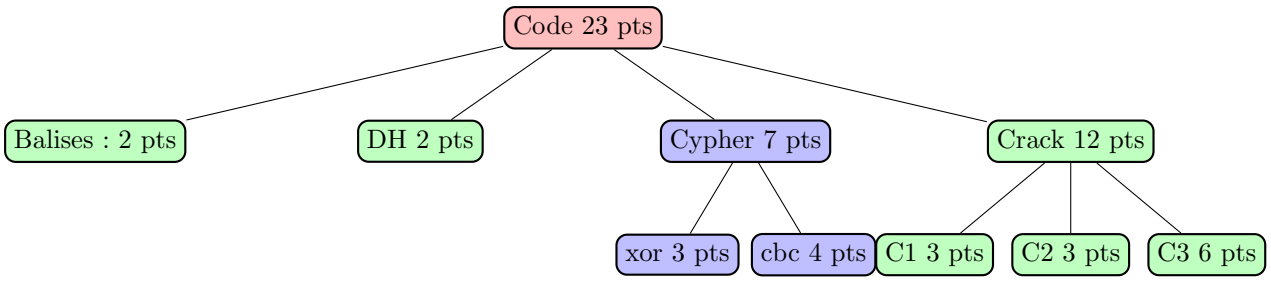
**Autre remarque :** Un fichier *dh_crack.h* sur Moodle donne des types et des prototypes de certaines fonctions. Utilisez-les ou pas suivant votre idée d'implémentation.

### 4.2 Attaque sur la clef : Pohlig-Hellman

Il n'y a rien à programmer dans cette partie. Un programme Python est donné qui implémente cette méthode et vous permet de tester les performances de cette méthode si vous avez la curiosité de l'essayer. Le programme calcule $a$ tel que $h = g^a$ connaissant $h$ et $g$. Il permet donc de calculer le nombre secret d'Alice, puis celui de Bob, et donc le nombre $(g^a)^b$ (ou $(g^b)^a$), c'est à dire la clef secrète.

## 5 Evaluation

### 5.1 Organisation globale du code et points associés (12 points)




où :
DH = programme de génération de la clef commune par la méthode de Diffie-Hellman.
Balises = gestion des balises et arguments des lignes de commandes.

### 5.2 Livrables de la partie Gestion de projet (8 pts)

Le contenu des livrables est développé dans le document "Aide à la gestion de projet" et dans la section suivante de ce document.

— Prévisionnel : 3 pts
— Doxygen : -1 si 0%, 1 pts si 50 +%
— Tests : 1 pts
— Bilan : 1 pts
— Validation : 2 pts

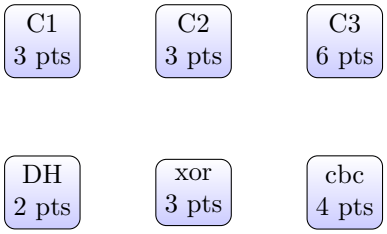
La note finale est la somme des deux notes. C'est cette unique note qui sera transmise à l'administration.

# 6 Tests, commentaires, livrables

## 6.1 Prévisionnel

La figure ci-dessous présente sous forme graphique les différents modules à programmer. Il y en a un peu trop pour une personne seule dans le temps imparti. C'est pourquoi vous allez devoir faire certains choix :

— indiquez les modules que vous aller implémenter et calculez le nombre maximal de points que peut vous rapporter ce choix
— étudiez la dépendance entre eux ou non des modules choisis
— faire un prévisionnel de l'organisation temporelle de votre code : en clair vous aller programmer vos choix dans quel ordre, quand, sans oublier que vous devez caser les tests. Les plus aventuriers pourront utiliser un outil comme GanttProject : http ://www.ganttproject.biz/ Sinon, on peut utiliser un tableur, ou tout simplement une feuille de papier et la prendre en photo.




Vous pouvez par exemple, relier les blocs choisis par des flèches dans la figure ci-dessus. Les flèches indiquent l'ordre de développement. Indiquez aussi le bloc de début.

Il manque le bloc "balises" qui est à faire de toutes façon.